PYTHON SUMMER PLAN · DAY 07

PYGAME PORT

Same physics. Different view.

Yesterday's smd_model.py imported untouched —
swap matplotlib for a 60 Hz game loop and a keyboard.

Sun · 17 May 2026 · ~3 hrs · Yuvraj Ruparel

07 · RECAP

Where Day 6 left off.

Day 6 — matplotlib version

- smd_model.py — pure physics: f(t, X), rk4_step, regime().
- smd_view.py — figure, sliders for m/k/c, RadioButtons for forcing.
- Live time + phase-space subplots.
- JSON config save/load via try/except (Day 5 muscle).
- matplotlib is fine for exploration — sluggish for real-time feel.

Day 7 — what changes

- Keep smd_model.py byte-for-byte identical. Import. Done.
- Replace the view layer with a pygame loop (60 Hz).
- Sliders → keyboard. +/− on m, k, c. [ ] on forcing freq.
- Render the mass using Day 3's Box from shapes.py.
- Spring drawn as a zigzag polyline; wall as a thick line.

💡 **If you touch smd_model.py today, you've broken the lesson.**

Model/view separation only proves itself when the model survives a view swap untouched.

07 · ARCHITECTURE

Model imported. Untouched.

```python
# smd_pygame.py  —  the new view layer

import pygame, numpy as np

# Same module you wrote yesterday.
# Don't open it. Don't edit it. Just call it.
from smd_model import f, rk4_step, regime, DEFAULT_CONFIG

# Day 3's shapes module — reuse Box for rendering.
from shapes import Box
from vector3d import Vector3D

# State, parameters, forcing — all the same shape as Day 6.
X = np.array([0.5, 0.0])  # [position, velocity]
m, k, c = 1.0, 10.0, 0.5
```

What this proves

- smd_model.py has no idea matplotlib or pygame exist.
- A model that depends on the view leaks into your tests.
- Same trick scales: Pong's ball physics can live in ball_model.py, rendered by pygame today, console-printed in unit tests tomorrow.
- If a Box from shapes.py can render here too, the abstraction was worth building.

💡 **Imports are a contract.**

Touch the model and you owe yourself a rewrite of the view. Don't.

Anatomy of a pygame loop.

```
pygame.init()
screen = pygame.display.set_mode((960, 540))
clock  = pygame.time.Clock()
dt = 1.0 / 60.0   # fixed simulation timestep

running, paused = True, False
while running:
    # 1. EVENTS  — keys, window close
    for event in pygame.event.get():
        handle_event(event)

    # 2. UPDATE  — advance physics one step
    if not paused:
        X = rk4_step(lambda t, x: f(t, x, m, c, k, F), t, X, dt)

    # 3. DRAW  — clear, render, flip
    screen.fill((15, 26, 46))
    draw_scene(screen, X, m, k, c)
    pygame.display.flip()
    clock.tick(60)   # cap at 60 FPS
```

Three phases, every frame

- EVENTS — drain the queue. Miss a frame and inputs stack up.
- UPDATE — one rk4_step per frame. If paused, skip update only — keep drawing.
- DRAW — clear the back-buffer, paint everything, flip().
- clock.tick(60) sleeps just enough to hold 60 FPS — gives you a stable dt = 1/60.
- Same loop you'll write for Pong tomorrow.

Keyboard as parameter knobs.

```
def handle_event(event):
    global running, paused, m, k, c, omega_f

    if event.type == pygame.QUIT:
        running = False

    elif event.type == pygame.KEYDOWN:
        k_ = event.key
        if   k_ == pygame.K_SPACE:    paused = not paused
        elif k_ == pygame.K_EQUALS:   m += 0.1
        elif k_ == pygame.K_MINUS:    m  = max(0.1, m - 0.1)
        elif k_ == pygame.K_k:        k += 0.5      # Shift+K to bump up
        elif k_ == pygame.K_c:        c += 0.1
        elif k_ == pygame.K_LEFTBRACKET:  omega_f = max(0.1, omega_f - 0.1)
        elif k_ == pygame.K_RIGHTBRACKET: omega_f += 0.1
        elif k_ == pygame.K_r:        X[:] = X0   # reset state
```

Control map

- SPACE — pause / resume (toggle).
- = / − — mass m up/down (0.1 step).
- K / k — spring k up/down. Wire Shift-K as decrement if you like.
- C / c — damping c up/down.
- [/] — forcing frequency ω_f.
- R — reset state X to initial conditions.
- ESC / window-close — quit (handle pygame.QUIT).

Reuse Day 3's Box. World → screen.

```
# World coords: x in meters, origin at wall, +x to the right.
# Screen coords: pixels, origin top-left, +y DOWN.

WORLD_TO_PX = 120   # 1 m = 120 px
WALL_X_PX   = 120
BASELINE_Y  = 280

# Day 3's Box, instantiated once and just repositioned each frame.
mass_box = Box(Vector3D(0, 0, 0), Vector3D(0, 0, 0), width=0.6, height=0.6)

def draw_scene(screen, X, m, k, c):
    pos = X[0]
    mass_box.pos.x = pos      # update model-side coords

    cx = WALL_X_PX + int(pos * WORLD_TO_PX)
    half = int(mass_box.width * WORLD_TO_PX / 2)
    pygame.draw.rect(screen, (61, 165, 255), (cx-half, BASELINE_Y-half, 2*half, 2*half))
```

Coord-system gotchas

- pygame y-axis points DOWN. matplotlib y-axis points UP. Negate or offset accordingly.
- Pick a scale once (px/m) and stick with it — magic numbers scattered everywhere kill debugging.
- Box owns width/height in meters; conversion to pixels lives in the view layer only.
- Draw the wall, spring, mass, then HUD text — in that order. Later layers cover earlier ones.

Zigzag spring as a polyline.

```
def draw_spring(screen, x_start, y, x_end, coils=12, amp=10):
    """Polyline zigzag from (x_start, y) to (x_end, y)."""
    pts = [(x_start, y)]
    span = x_end - x_start
    for i in range(1, coils + 1):
        px = x_start + span * i / (coils + 1)
        py = y + (amp if i % 2 else -amp)
        pts.append((px, py))
    pts.append((x_end, y))
    pygame.draw.lines(screen, (200, 210, 225), False, pts, 2)

# Wall — single thick vertical line.
pygame.draw.line(screen, (160, 175, 195),
                 (WALL_X_PX, BASELINE_Y-80),
                 (WALL_X_PX, BASELINE_Y+80), 4)
```

Why a polyline

- pygame.draw.lines takes a list of points — no need for individual segments.
- Coil count fixed, but x_end moves each frame (= mass center − half-width).
- Amplitude is constant, span varies → coils visually stretch and compress. Looks right.
- Want fancier? Sinusoidal interpolation: py = y + amp * sin(2π · i / coils). Save for polish.

💡 **Procedural drawing beats sprites for parametric objects.**

A spring isn't a fixed-size asset — its length changes every frame. A polyline regenerated per frame is the right primitive.

Fixed dt and a live readout.

```python
# Render text overlay — current parameters + damping regime.
font = pygame.font.SysFont("menlo", 16)

def draw_hud(screen, m, k, c, omega_f, paused):
    lines = [
        f"m = {m:5.2f} kg     [= / − to change]",
        f"k = {k:5.2f} N/m    [k / K to change]",
        f"c = {c:5.2f} N·s/m  [c / C to change]",
        f"ω_f = {omega_f:5.2f}  [[ / ] to change]",
        f"regime: {regime(m, c, k)}",
        f"{'PAUSED' if paused else 'RUNNING'}",
    ]
    for i, ln in enumerate(lines):
        surf = font.render(ln, True, (220, 230, 245))
        screen.blit(surf, (20, 20 + i * 22))
```

Fixed dt = 1/60 — when to care

- clock.tick(60) targets 60 Hz; if the frame is heavy, you might drop to 30 → physics half-speed.
- For SMD, single rk4_step per frame is fine — no risk of drift at this scale.
- Real fix is the accumulator pattern: integrate N substeps until you've consumed real elapsed time. Save for Pong if you need it.
- HUD refresh every frame is cheap — just render() and blit().

TODAY'S EXERCISE · 180 MIN

Spring-mass-damper, pygame edition.

Build smd_pygame.py — imports yesterday's smd_model.py and Day 3's shapes.py, both untouched.

IMPORT, DON'T REWRITE

Reuse

- smd_model.py — import f, rk4_step, regime, DEFAULT_CONFIG. Do not edit.
- shapes.py + vector3d.py — import Box and Vector3D for the mass render.
- If you find yourself adding a pygame import inside smd_model.py, stop.

LOOP & INPUT

View skeleton

- pygame.init(), 960×540 window, Clock at 60 FPS.
- Event handler: QUIT, KEYDOWN for SPACE, =/−, k/K, c/C, [/], R.
- One rk4_step per frame when not paused. Skip update only — keep drawing.
- fill(navy bg), draw scene, draw HUD, display.flip().

RENDER

Scene

- Wall — thick vertical line at the left.
- Spring — polyline zigzag from wall to mass-left-edge.
- Mass — rect derived from Box(width=0.6, height=0.6).
- HUD — m, k, c, ω_f, regime, paused state. Top-left, Menlo 16.

Hard rule: open smd_model.py only to read it. If you opened it to edit, the lesson was lost.

Things that will bite.

⚠ Event drain every frame.

pygame.event.get() must run every frame or the OS thinks your window hung. Even if you don't care about most events, drain the queue.

⚠ Pause skips update, NOT draw.

If you put display.flip() inside the not-paused branch, the screen freezes on the last drawn frame and the HUD won't update. Draw every frame, integrate only when running.

💡 y-axis flip is easy to miss.

pygame y grows downward. If the mass moves up when it should move down, you forgot the negation. Centralize the world→screen mapping in one helper, not scattered.

💡 Same Box object every frame.

Don't construct a new Box inside draw_scene — that's a per-frame allocation. Build once at startup, mutate .pos.x in place.

WRAP-UP · WHAT THIS FEEDS

Today → next week → Pong.

WEEK 1 · DONE

OOP foundation

Vector3D, Shape/Circle/Box, exceptions, RK4, model/view separation. The Week-1 toolkit is complete. Everything from here imports it.

MON DAY 08 · pygame core

Loop fluency

Tomorrow you'll rewrite today's loop without the SMD physics — just bouncing rectangles. Goal: own the loop / events / draw cycle before touching sprites.

WEEK 2 · BUILD

Pong, proper

By Sat you'll have Pong with paddles, ball, scoring, simple AI. The loop pattern is identical to today's. The ball's update step is just rk4 with no spring force.

If smd_model.py is still untouched in the diff, the lesson stuck. Verify before closing.